INTELLIGENT ROBOT NAVIGATION USING REINFORCEMENT LEARNING IN DYNAMIC ENVIRONMENTS

MINI PROJECT REPORT

Student Name(s): _Devanshu Sharma_____
Reg. Number: __23FE10CSE00274_____
Course / Subject: __Reinforcement Learning_____
Faculty Name: __Ms. Neha_____
Department / Institution: __Manipal University Jaipur___
Academic Year: __2026-27_____

Abstract

Autonomous robot navigation in dynamic environments is challenging because obstacles may be fixed or move over time. This project develops an intelligent grid-based robot navigation system using tabular Q-Learning. The objective is to train an agent in procedurally generated environments and evaluate whether its learned policy can navigate layouts it has not encountered during training. A 10 × 10 grid environment represents the robot, goal, static obstacles and horizontally moving obstacles. The agent observes the relative direction of the goal and the condition of its eight neighbouring cells, and selects one of four actions: up, down, left or right. Training uses an ε-greedy policy and reward-based Q-value updates. The trained Q-table is saved and later loaded into a Pygame interface, where a user can design a custom obstacle layout and test the policy without further learning. In the reported experiment, the agent achieved a 76.00% success rate over 5,000 training episodes and a 98.20% success rate over 500 separately generated evaluation scenarios. These results indicate promising performance within the tested scenario distribution; they do not guarantee success on every arbitrary custom map.

1. Introduction

Robot navigation is a fundamental problem in autonomous systems. A robot must select actions that move it toward a destination while avoiding obstacles and responding to changes in its surroundings. Static obstacles remain in fixed locations, whereas dynamic obstacles can change position and make a previously safe route unsuitable.

The motivation for this project is to explore how an agent can learn navigation behaviour through interaction rather than relying only on a manually programmed route. The problem is to develop and test a Q-Learning-based agent that learns in multiple generated grid layouts and can subsequently be evaluated in a user-created environment without retraining.

The main objectives are: (i) implement a grid-based environment with static and moving obstacles; (ii) train a tabular Q-Learning agent over procedurally generated scenarios; (iii) evaluate the saved policy on new generated scenarios; and (iv) provide a Pygame interface for custom environment creation and navigation testing.

2. Methodology

The project uses tabular Q-Learning, an off-policy, model-free reinforcement learning algorithm. At each time step, the agent observes state s, selects action a, receives reward r and observes the next state s′. The Q-value is updated during training using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

For terminal transitions, the target is the immediate reward. An ε-greedy policy selects random actions with probability ε during early training and increasingly favours actions with higher estimated Q-values as ε decays.

2.1 Environment and State Space

The environment is a 10 × 10 discrete grid. The robot begins at the top-left cell and the goal is selected at an unoccupied cell. Static obstacles occupy blocked cells. Moving obstacles travel horizontally within defined row boundaries and reverse direction when blocked or when reaching a boundary. The state encoder produces a 10-element tuple: two values indicating the vertical and horizontal direction toward the goal, followed by eight local cell indicators for the surrounding neighbourhood. A cell indicator represents free space, a static/out-of-grid obstacle, or a moving obstacle with its direction.

2.2 Action Space and Reward Function

The action space contains four discrete actions: 0—up, 1—down, 2—left and 3—right. The reward design encourages reaching the goal efficiently and discourages invalid movement and collisions.

Event	Reward
Each valid step	−1
Move outside grid / into static obstacle	−5
Collision with moving obstacle	−20
Reach the goal	+100

Table 1. Reward function used by the environment.

2.3 Hyperparameters and Training Strategy

Parameter	Value
Learning rate (α)	0.10
Discount factor (γ)	0.95
Initial ε / minimum ε	1.00 / 0.05

ε decay per episode	0.999
Training episodes	5,000
Maximum steps per episode	200

Table 2. Q-Learning hyperparameters.

2.4 Training, Evaluation and Software

For each training episode, a new scenario is generated with a fixed grid size and configured numbers of static and moving obstacles. A breadth-first search check verifies that a path exists when considering static obstacles. The agent interacts with the environment, updates its Q-table and decays ε after each episode. The resulting Q-table is saved as a pickle file. Evaluation loads this saved table and uses a greedy policy without calling the learning update. The Q-table lookup was also adjusted so unseen states do not create new entries during inference.

The evaluation experiment used 500 newly generated scenarios from the same scenario-generation procedure. These are unseen layouts, but they are drawn from the same general distribution as the training layouts. The Pygame interface separately allows a user to place static and moving obstacles and select a goal before testing the saved policy. Performance on arbitrary user-created layouts must be measured separately.

The implementation uses Python 3.12.2, Pygame 2.6.1 for interactive visualisation, and Matplotlib for plotting training and evaluation results. NumPy and Python's standard libraries support the computational workflow and data handling.

2.5 System Architecture and Workflow

The system is organised into five main components: (1) scenario generator, (2) GridWorld environment, (3) state encoder, (4) Q-Learning agent and saved Q-table, and (5) Pygame interface and evaluation/plotting modules. During training, generated scenarios are passed to the environment; encoded states are processed by the agent; and the learned Q-table is saved. During custom testing, the interface converts the user's layout into an environment scenario, loads the saved Q-table and runs the policy without updating it.

INSERT FIGURE 1 HERE: System architecture / workflow diagram
Suggested flow: Scenario Generator → GridWorld → State Encoder → Q-Learning Agent → Saved Q-table
Custom Pygame Layout → GridWorld → State Encoder → Frozen Policy → Robot Navigation

Figure 1. Overall architecture and training/evaluation workflow.

3. Experimental Results

The agent completed 5,000 training episodes. It succeeded in 3,800 episodes, giving a training success rate of 76.00%; 852 episodes ended in collision (17.04%) and 348 timed out (6.96%). The average reward was 4.81 and the average episode length was 52.19 steps.

On 500 separately generated evaluation scenarios, the agent succeeded in 491 episodes (98.20%), recorded no collisions (0.00%) and timed out in 9 episodes (1.80%). Average evaluation reward was 77.37 and average episode length was 21.41 steps. The evaluation success rate is higher than the training success rate; this should be interpreted in the context of the generated scenario distribution and the difference between exploratory training and greedy evaluation.

3. Experimental Results (continued)

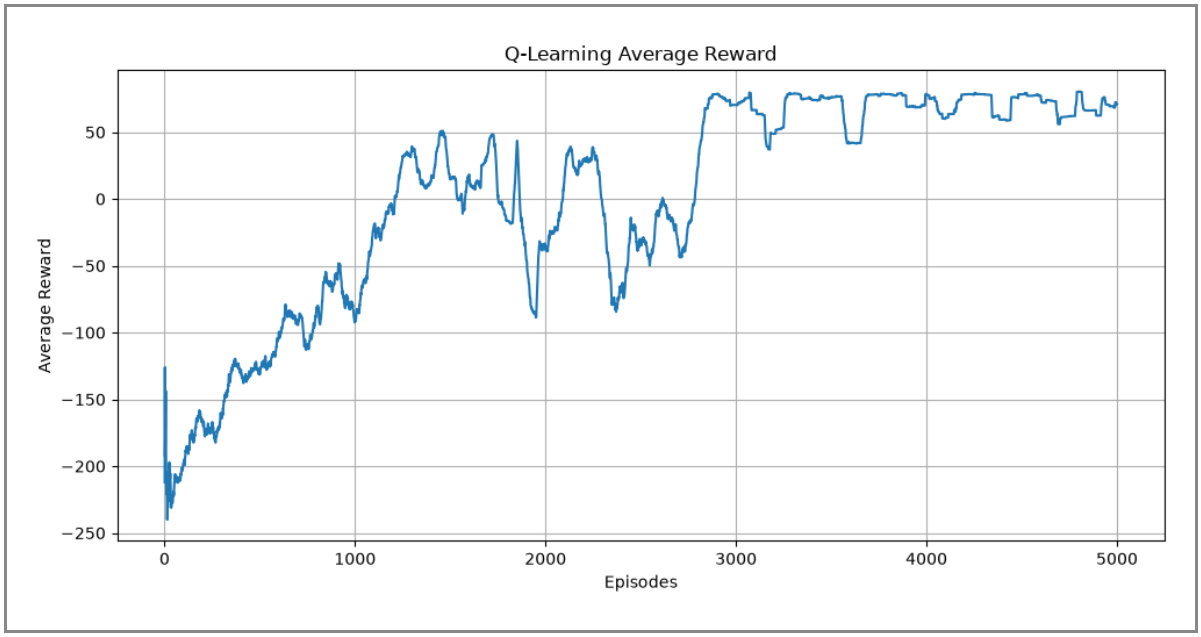
Metric	Training	Evaluation
Episodes	5,000	500
Successes	3,800	491
Success rate	76.00%	98.20%
Collisions	852 (17.04%)	0 (0.00%)
Timeouts	348 (6.96%)	9 (1.80%)
Average reward	4.81	77.37
Average steps	52.19	21.41

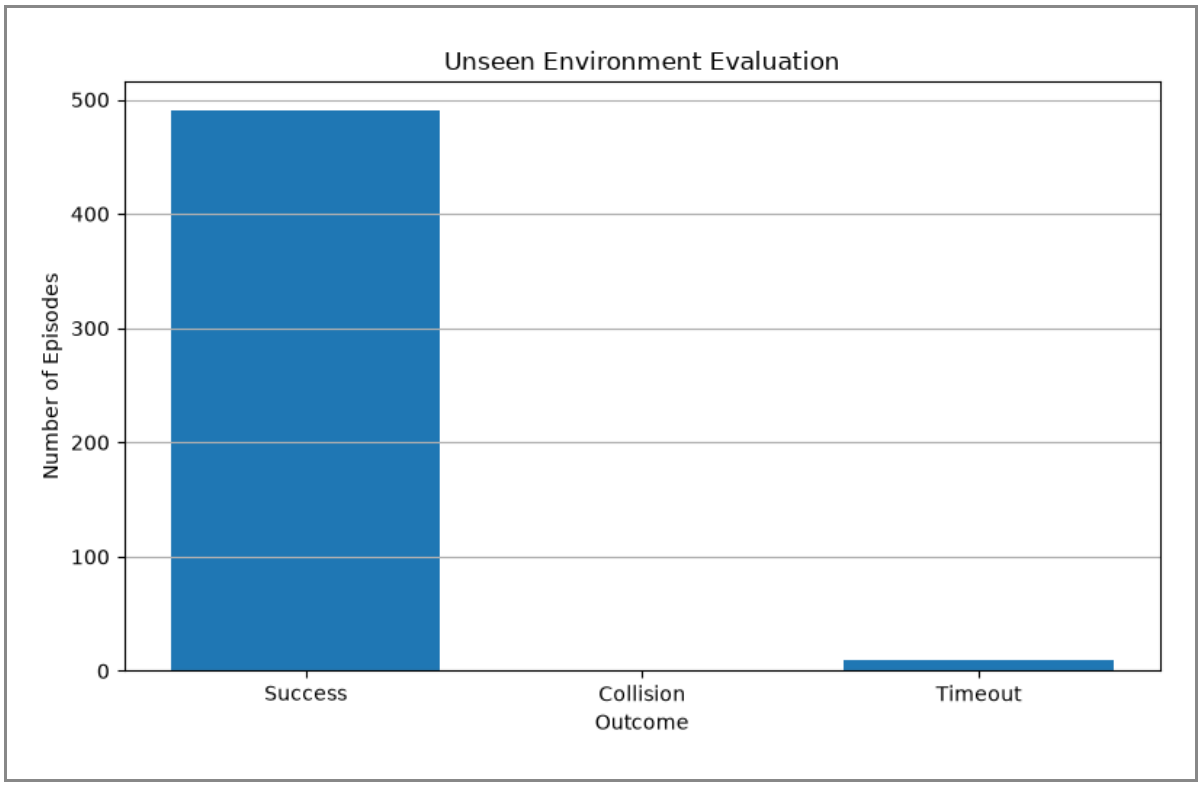
Table 3. Training and generated-scenario evaluation performance.

A separate baseline comparison was not performed in the current experiments. Therefore, no baseline score is claimed. A suitable future comparison would evaluate the learned policy against a random-action agent and a simple goal-directed heuristic under identical scenario seeds and step limits.

Insert the generated plots and screenshots in the spaces below. Use the original PNG files from the project's results/graphs folder. For screenshots, capture the Pygame environment during setup and during robot navigation.







4. Conclusion

This project implemented a tabular Q-Learning system for grid-based robot navigation with static and moving obstacles. The agent was trained across procedurally generated scenarios and evaluated using a saved, frozen policy. The measured 98.20% success rate on 500 unseen generated layouts demonstrates promising generalisation within the tested scenario

distribution. The Pygame interface provides a practical way to create custom layouts and observe the learned policy.

The approach has limitations. The compact local state may represent different global situations identically, and the evaluation scenarios come from the same generator family as training. Static-path validation does not guarantee a collision-free route in the presence of moving obstacles. Future work can include richer state representations, broader scenario distributions, explicit baseline comparisons, more diverse obstacle trajectories and systematic testing on manually designed maps.

References

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, pp. 279–292, 1992.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[3] Python Software Foundation, "Python 3 Documentation." [Online]. Available: https://docs.python.org/3/

[4] Pygame Community, "Pygame Documentation." [Online]. Available: https://www.pygame.org/docs/

[5] Matplotlib Development Team, "Matplotlib Documentation." [Online]. Available: https://matplotlib.org/stable/

Before submission: replace all student/faculty placeholders, insert the actual figures and screenshots, and verify the institution's preferred title-page details.